Content

이번 주차의 학습목표와 학습내용을 확인하세요.

01. 큐(Queue)

선형 큐와 원형 큐의 구조와 동작 원리를 이해하고 설명할 수 있다.

02. 덱(Deque)

덱의 개념과 동작 원리를 이해하고 설명할 수 있다.

03. 우선순위 큐

우선순위 큐의 개념과 동작 원리를 이해하고 설명할 수 있다.

큐(Queue)

큐의 개념과 동작 원리

큐의 개념

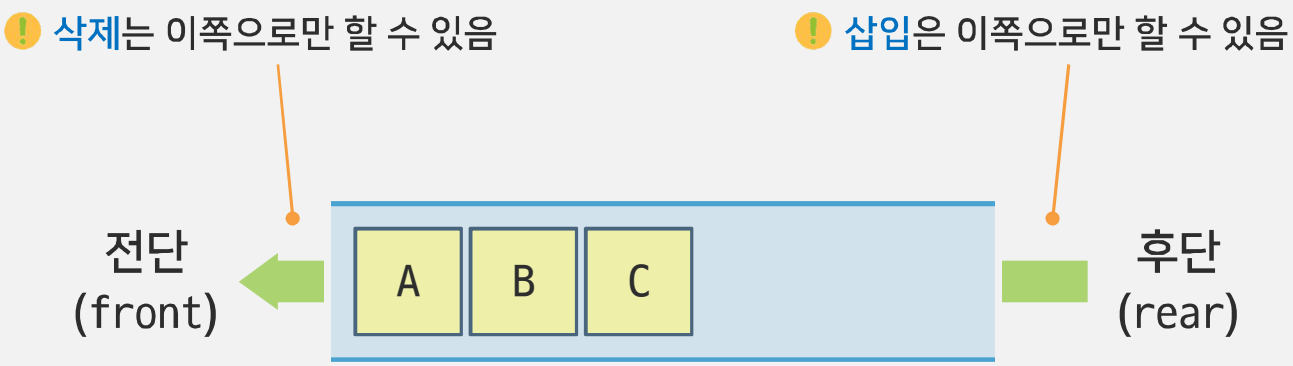
선입선출(FIFO, First-In First-Out)의 자료구조

<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

큐의 개념과 동작 원리

큐의 구조

1. 삽입과 삭제는 FIFO순서를 따름
2. 삽입은 큐의 후단에서, 삭제는 전단에서 이루어짐

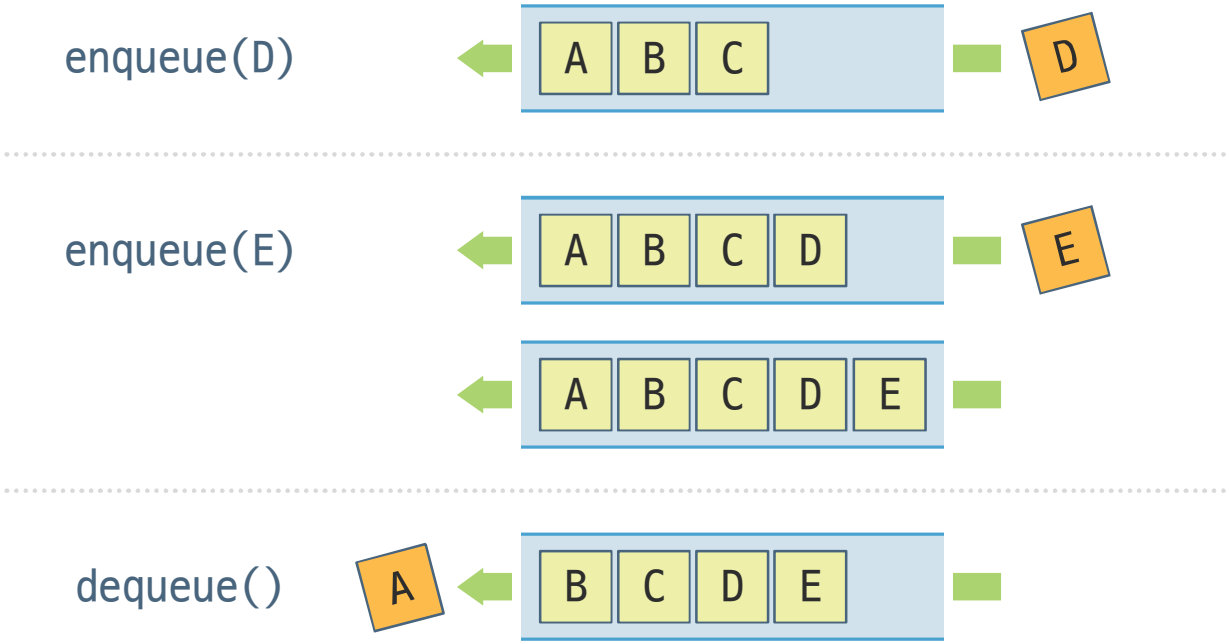


큐의 개념과 동작 원리

큐의 연산

삽입	enqueue()

삭제	dequeue()



큐의 개념과 동작 원리

큐 ADT

데이터

- 선입선출(FIFO)의 접근 방법을 유지하는 항목들의 모음

연산

- Queue(): 비어 있는 새로운 큐를 만듦
- isEmpty(): 큐가 비어있으면 True를 아니면 False를 반환함
- enqueue(x): 항목 x를 큐의 맨 뒤에 추가함
- dequeue(): 큐의 맨 앞에 있는 항목을 꺼내 반환함
- peek(): 큐의 맨 앞에 있는 항목을 삭제하지 않고 반환함
- size(): 큐의 모든 항목들의 개수를 반환함
- clear(): 큐를 공백상태로 만듦

큐의 개념과 동작 원리

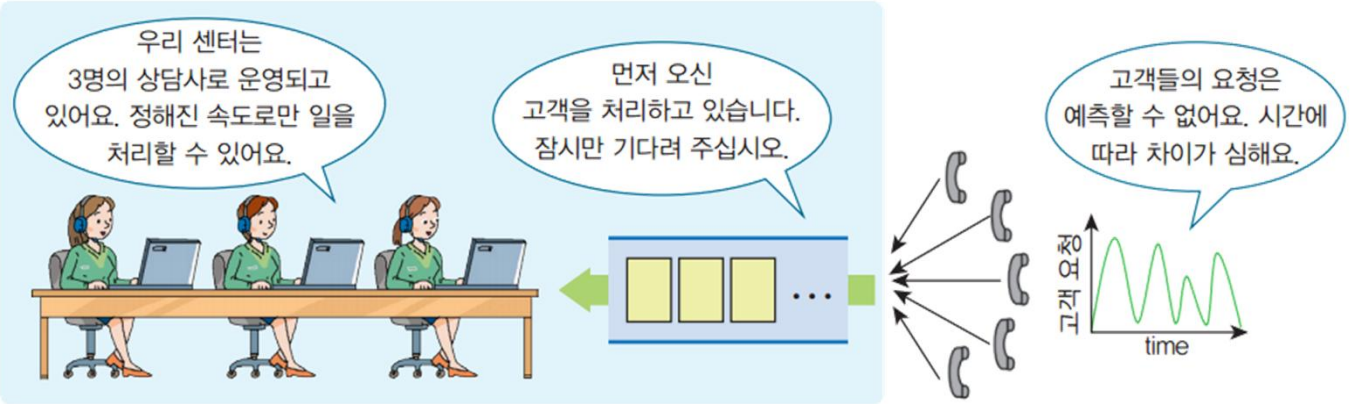
큐의 예시

서비스 센터,
은행 콜, 공항
비행기 등의 대기열

프린터와 컴퓨터
사이의 인쇄 작업 큐
(버퍼링)

실시간 비디오
스트리밍에서의
버퍼링

통신에서의
데이터 패킷들의
모델링에 이용



<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

원형 큐

선형 큐의 비효율성

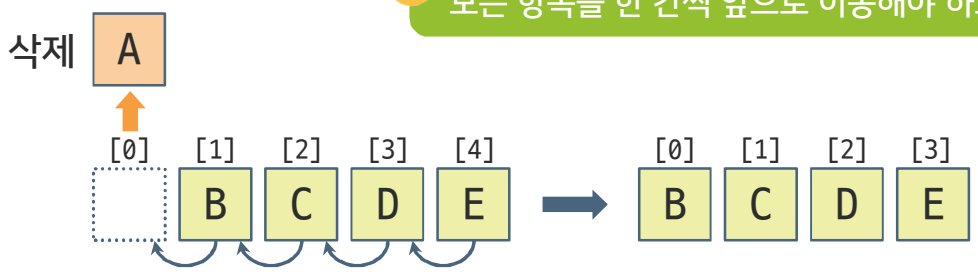
01 삽입연산 복잡도 (enqueue(item)) : $O(1)$

```
def enqueue(item):
    items.append(item)              # 리스트의 맨 뒤에 items 추가
```

02 삭제연산 복잡도 (dequeue()) : $O(n)$

```
def dequeue():
    if not isEmpty():               # 공백상태가 아니면
        return items.pop(0)         # 맨 앞 항목을 꺼내서 반환
```

리스트의 맨 앞에서 항목을 삭제하면 그 항목 이후의
모든 항목을 한 칸씩 앞으로 이동해야 하므로 매우 비효율적임



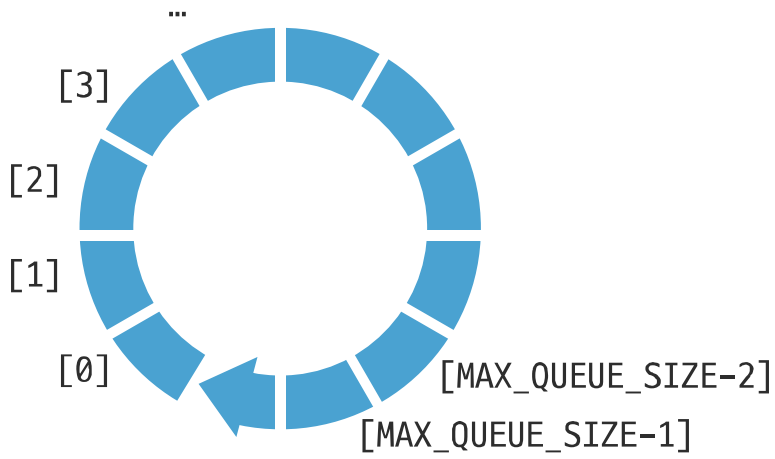
원형 큐

원형 큐의 효율성

01 배열을 원형으로 사용

02 전단과 후단을 위한 2개의 변수
- front: 첫번째 요소 하나 앞의 인덱스
- rear: 마지막 요소의 인덱스

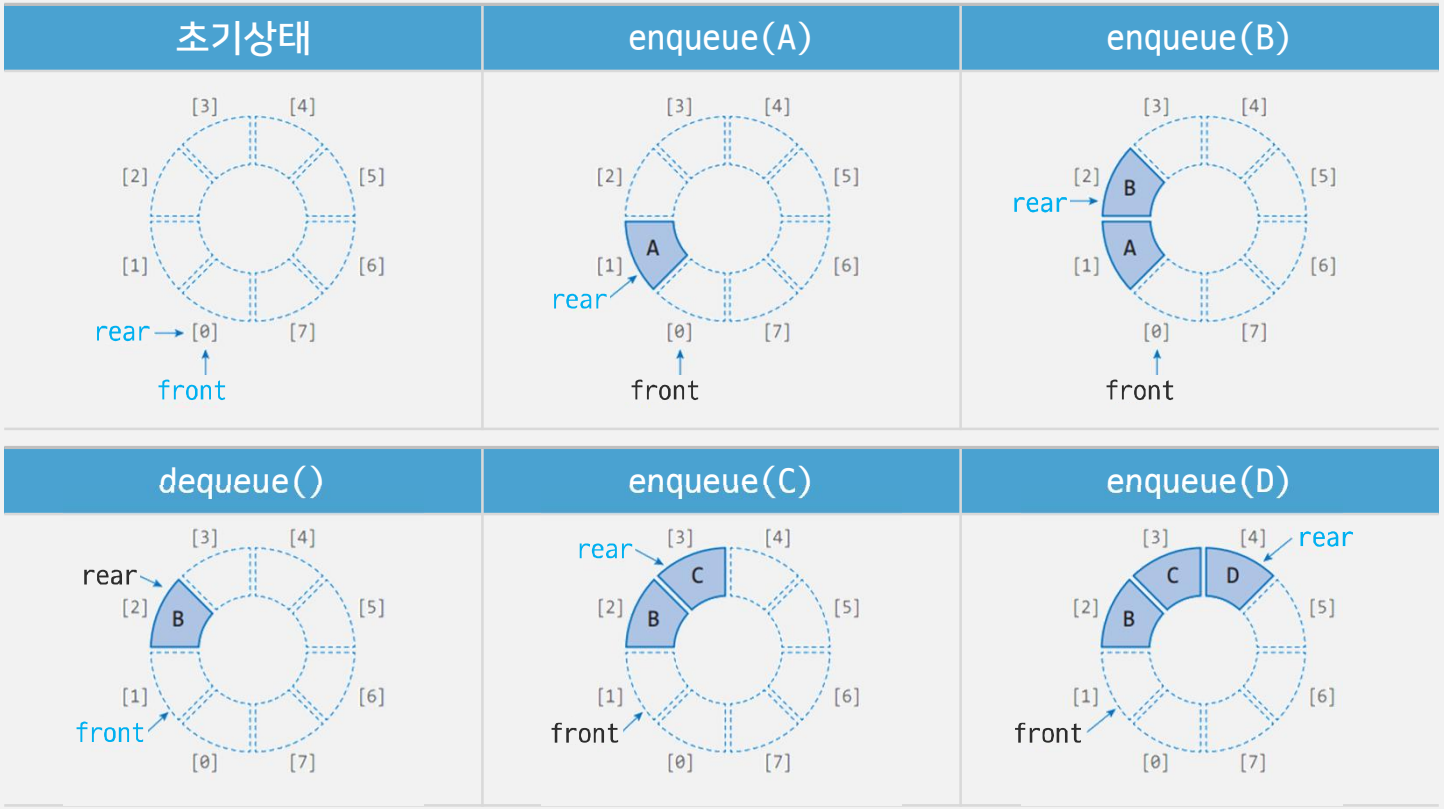
03 회전(시계방향) 방법
- front ← (front+1) % MAX_QSIZE
- rear ← (rear+1) % MAX_QSIZE



원형 큐

원형 큐의 개념

원형 큐의 삽입과 삭제



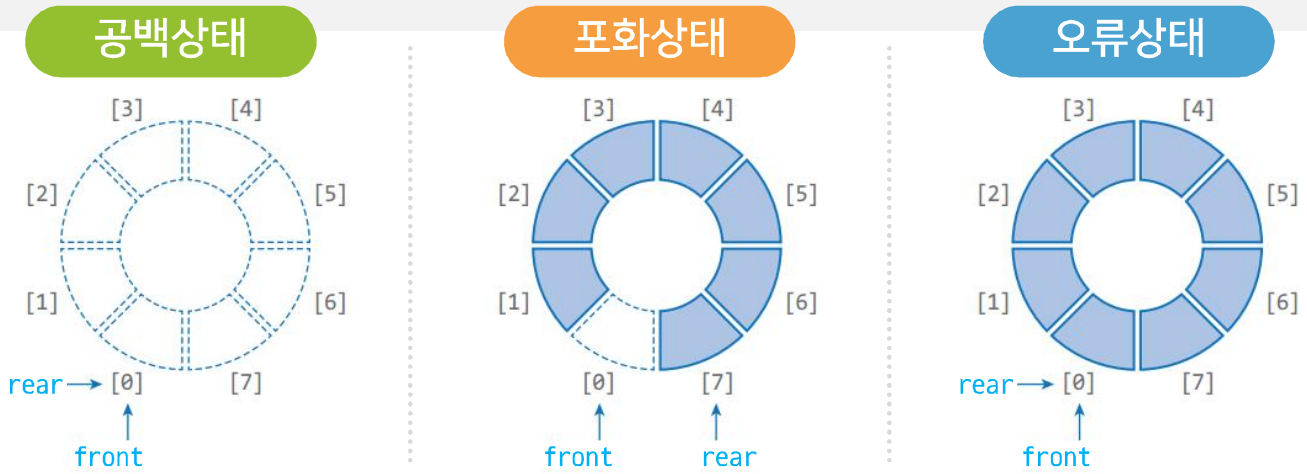
원형 큐

원형 큐의 개념

공백상태와 포화상태

공백상태	포화상태
`front == rear`	`front % M == (rear+1) % M`

- 공백상태와 포화상태 구별 ➡ 하나의 공간은 항상 비워둠



원형 큐

원형 큐의 구현

리스트의 크기 미리 결정 ➡ 포화상태 존재

```
MAX_QSIZE = 10                              # 원형 큐의 크기
class CircularQueue :
    def __init__( self ) :                  # CircularQueue 생성자
        self.front = 0                      # 큐의 전단 위치
        self.rear = 0                       # 큐의 후단 위치
        self.items = [None] * MAX_QSIZE     # 항목 저장용 리스트 [None,None,…]
    def isEmpty( self ) : return self.front==self.rear
    def isFull( self ) : return self.front == (self.rear+1)%MAX_QSIZE
    def clear(self): self.front = self.rear
```

원형 큐

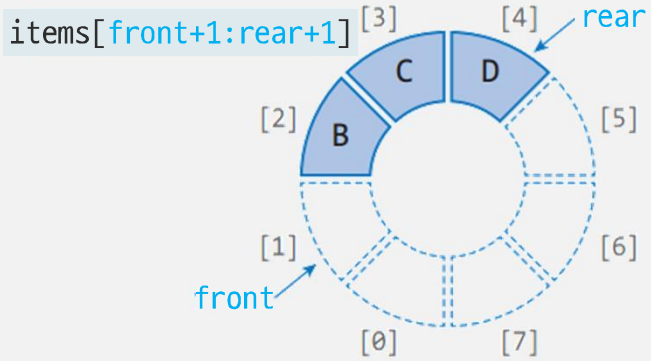
원형 큐의 연산

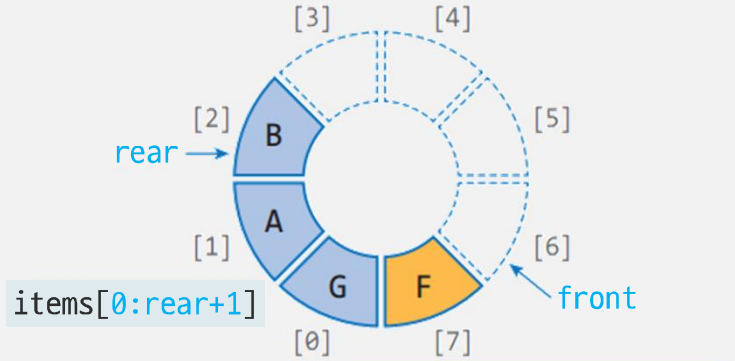
```
def enqueue( self, item ):
    if not self.isFull():                      # 포화상태가 아니면
       self.rear = (self.rear+1)%MAX_QSIZE     # rear 회전
       self.items[self.rear] = item            # rear 위치에 삽입

def dequeue( self ):
    if not self.isEmpty():                     # 공백상태가 아니면
       self.front = (self.front+1)%MAX_QSIZE   # front 회전
       return self.items[self.front]           # front 위치의 항목 반환

def peek( self ):
    if not self.isEmpty():
       return self.items[(self.front + 1)%MAX_QSIZE]

def size( self ) :
    return (self.rear - self.front + MAX_QSIZE)%MAX_QSIZE
```

원형 큐

원형 큐의 연산

```
def display( self ):
    out = []
    if self.front < self.rear :
       out = self.items[self.front+1:self.rear+1]     # 슬라이싱
    else:
       out = self.items[self.front+1:MAX_QSIZE] \     # 다음 줄에 계속...
          + self.items[0:self.rear+1]                 # 슬라이싱
print("[f=%s,r=%d] ==> "%(self.front, self.rear), out)
```





(2/2)

원형 큐

테스트 코드

```
q = CircularQueue()                    # 원형 큐 만들기(MAX_QSIZE=10)
for i in range(8): q.enqueue(i)        # 0, 1, …, 7 삽입(f=0, r=8)
q.display()                            # 원형 큐에서 구현한 print()호출
for i in range(5): q.dequeue();        # 5번 삭제(f=5, r=8)
q.display()
for i in range(8,14): q.enqueue(i)     # 8, 9, …, 13 삽입(f=5, r=4)
q.display()
```

C:\WINDOWS\system32\cmd.exe

```
[f=0,r=8] ==> [0, 1, 2, 3, 4, 5, 6, 7]
[f=5,r=8] ==> [5, 6, 7]
[f=5,r=4] ==> [5, 6, 7, 8, 9, 10, 11, 12, 13]
```

원형 큐

파이썬의 큐

큐(Queue)와 스택(LifoQueue) 클래스 제공

- 큐 모듈 import
- 객체생성
- 삽입 put()
- 삭제 get()

```
import queue                    # 파이썬의 큐 모듈 포함
Q = queue.Queue(maxsize=20)     # 큐 객체 생성(최대크기 20)
for v in range(1, 10) :
    Q.put(v)
print("큐의 내용: ", end='')
for _ in range(1, 10) :
    print(Q.get(), end=' ')
print()
```

C:\WINDOWS\system32\cmd.exe

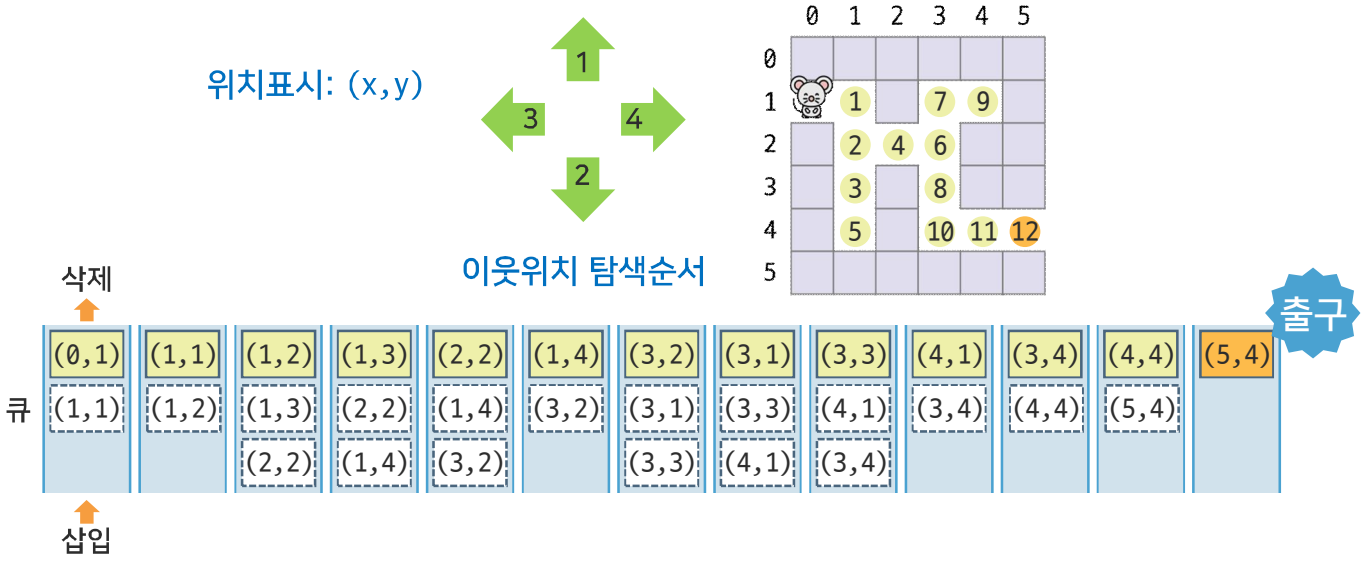
```
큐의 내용: 1 2 3 4 5 6 7 8 9
```

큐의 응용

너비우선탐색

미로 탈출 방법으로 너비우선탐색 이용

- 큐에 가능한 다음 위치를 순서대로 저장
- 막다른 길을 만나면 가장 먼저 저장된 위치를 다시 시도
- 입구에서 가장 가까운 위치를 먼저 탐색



큐의 응용

너비우선탐색

알고리즘

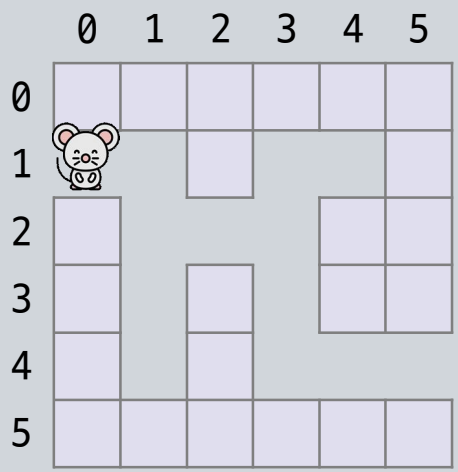
```
def BFS():                                  # 너비우선탐색 함수
    que = CircularQueue()
    que.enqueue((0,1))
    print('BFS: ')                          # 출력을 'BFS'로 변경

    while not que.isEmpty():
        here = que.dequeue()
        print(here, end='->')
        x,y = here
        if(map[y][x] == 'x'): return True
        else:
            map[y][x] = '.'
            if isValidPos(x, y - 1) : que.enqueue((x, y - 1))    # 상
            if isValidPos(x, y + 1) : que.enqueue((x, y + 1))    # 하
            if isValidPos(x - 1, y) : que.enqueue((x - 1, y))    # 좌
            if isValidPos(x + 1, y) : que.enqueue((x + 1, y))    # 우
    return False
```

큐의 응용

너비우선탐색

테스트코드

```
map = [ ['1','1','1','1','1','1'],
        ['e','0','1','0','0','1'],
        ['1','0','0','0','1','1'],
        ['1','0','1','0','1','1'],
        ['1','0','1','0','0','x'],
        ['1','1','1','1','1','1']]
MAZE_SIZE = 6
result = BFS()
if result : print('--> 미로탐색 성공')
else : print('-->미로탐색 실패')
```



C:\WINDOWS\system32\cmd.exe

너비우선탐색 과정

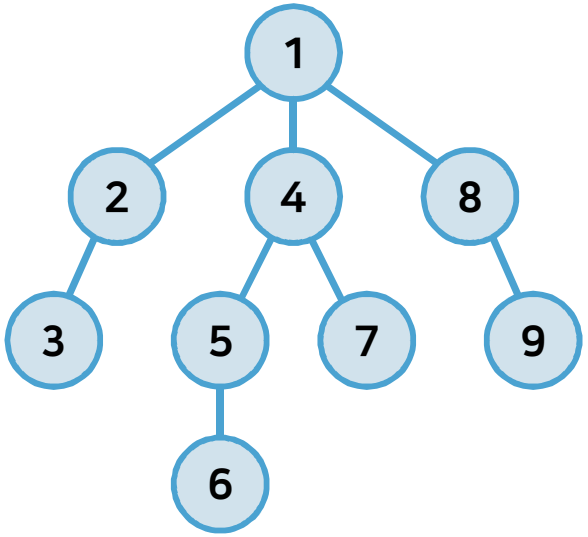
```
BFS
(0, 1)->(1, 1)->(1, 2)->(1, 3)->(2, 2)->(1, 4)->(3, 2)->(3, 1)->(3, 3)->(4, 1)->
(3, 4)->(4, 4)->(5, 4)-> --> 미로탐색 성공
```

큐의 응용

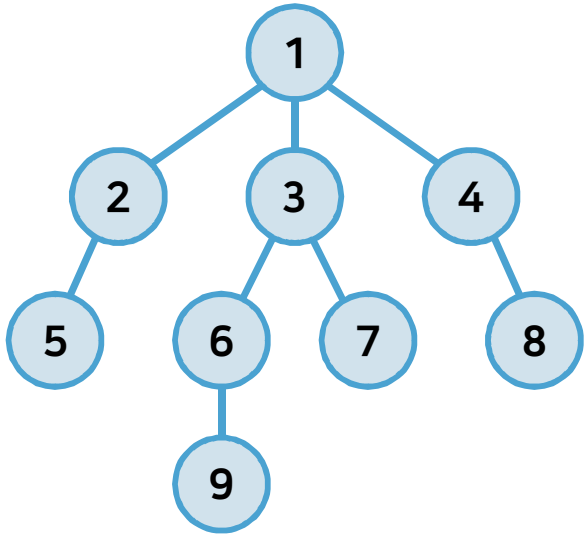
너비우선탐색

깊이우선탐색 vs 너비우선탐색

DFS



BFS



덱(Deque)

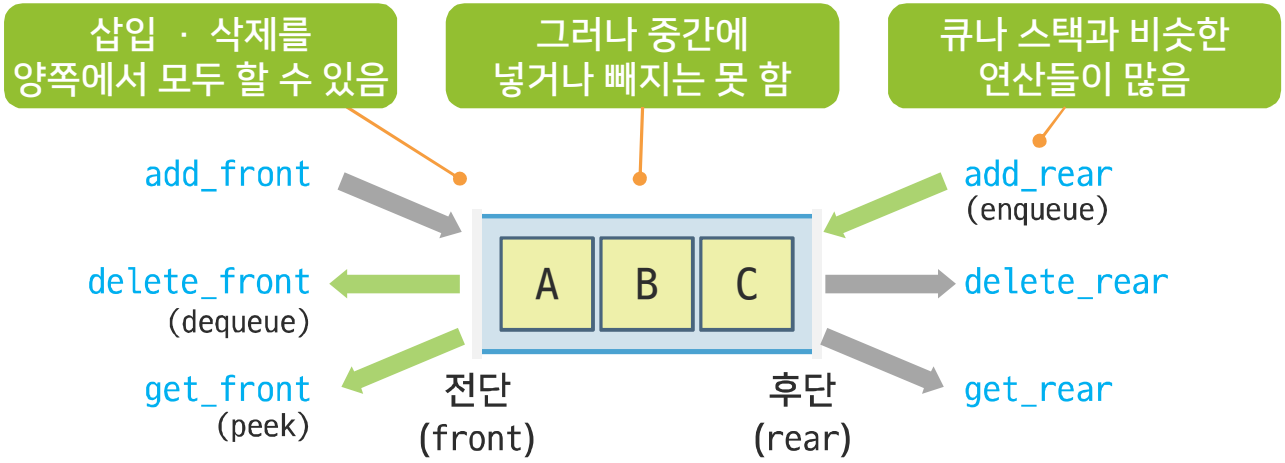
덱의 개념과 동작 원리

덱의 개념

덱

/ deque, double-ended queue /

- 스택이나 큐보다 입출력이 자유로운 자료구조
- 전단(front)와 후단(rear)에서 모두 삽입과 삭제가 가능



덱의 개념과 동작 원리

덱 ADT

데이터

- 전단과 후단을 통한 접근을 허용하는 항목들의 모음

연산

- Deque(): 비어 있는 새로운 덱을 만듦
- isEmpty(): 덱이 비어있으면 True를 아니면 False를 반환함
- addFront(x): 항목 x를 덱의 맨 앞에 추가함
- deleteFront(): 맨 앞의 항목을 꺼내서 반환함
- getFront(): 맨 앞의 항목을 꺼내지 않고 반환함
- addRear(x): 항목 x를 덱의 맨 뒤에 추가함
- deleteRear(): 맨 뒤의 항목을 꺼내서 반환함
- getRear(): 맨 뒤의 항목을 꺼내지 않고 반환함

덱의 개념과 동작 원리

덱 ADT

연산

- `isFull()`:
  덱이 가득 차 있으면 True를 아니면 False를 반환함
- `size()`: 덱의 모든 항목들의 개수를 반환함
- `clear()`: 덱을 공백상태로 만듦

덱의 개념과 동작 원리

덱의 연산

원형 덱의 연산

- 큐와 데이터는 동일하고 연산은 유사
- addRear(), deleteFront(), getFront()
  - 큐의 enqueue, dequeue, peek 연산과 동일
- addRear(), deleteRear(), getRear()
  - 덱의 후단(rear)을 스택의 상단(top)으로 사용하면, 스택의 push, pop, peek 연산과 동일

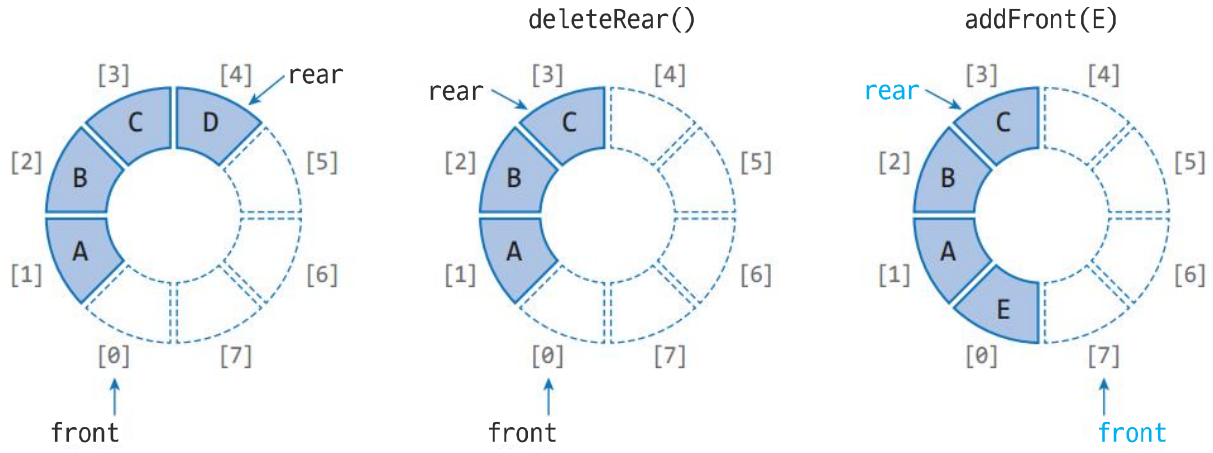
덱의 개념과 동작 원리

덱의 연산

원형 큐에서 추가된 원형 덱의 연산

- deleteRear(), addFront(), getRear()
  - 반 시계방향 회전 필요

```
front ← (front-1 + MAX_QSIZE) % MAX_QSIZE
rear ← (rear-1 + MAX_QSIZE) % MAX_QSIZE
```



덱의 구현

클래스 상속을 이용한 구현

1. 원형 큐를 상속하여 원형 덱 클래스를 구현

```
class CircularDeque(CircularQueue) :     # CircularQueue에서 상속
```

2. 덱의 생성자는 상속되지 않음

- front, rear, items 등 멤버변수는 추가선언 안함
- 자식클래스에서 부모 부르는 함수는 super()

```
def __init__( self ) :              # CircularDeque의 생성자
    super().__init__()              # 부모 클래스의 생성자를 호출함
```

덱의 구현

클래스 상속을 이용한 구현

3 재사용 멤버

- isEmpty, isFull, size, clear

4 인터페이스 변경 멤버들

```
def addRear( self, item ): self.enqueue(item)        # enqueue 호출
def deleteFront( self ): return self.dequeue()       # 반환에 주의
def getFront( self ): return self.peek()             # 반환에 주의
```

덱의 구현

클래스 상속을 이용한 구현

5 추가 구현 메소드

```
def addFront( self, item ):                    # 새로운 기능 : 전단 삽입
    if not self.isFull():
       self.items[self.front] = item           # 항목 저장
       self.front = self.front - 1             # 반시계 방향으로 회전
       if self.front < 0: self.front = MAX_QSIZE -1

def deleteRear( self ):                        # 새로운 기능 : 후단 삭제
    if not self.isEmpty():
       item = self.items[self.rear];           # 항목 복사
       self.rear = self.rear - 1               # 반시계 방향으로 회전
       if self.rear < 0: self.rear = MAX_QSIZE - 1
       return item                             # 항목 반환

def getRear( self ):                           # 새로운 기능 : 후단 peek
    return self.items[self.rear]
```

덱의 구현

클래스 상속을 이용한 구현

테스트코드

```
dq = CircularDeque()                    # 덱 객체 생성. f=r=0
for i in range(9):                      # i: 0, 1, 2, …, 8
    if i%2==0 : dq.addRear(i)           # 짝수는 후단에 삽입
    else : dq.addFront(i)               # 홀수는 전단에 삽입
dq.display()                            # front=6,rear=5
for i in range(2): dq.deleteFront()     # 전단에서 두 번의 삭제: f=8
for i in range(3): dq.deleteRear()      # 후단에서 세 번의 삭제: r=2
dq.display()
for i in range(9,14): dq.addFront(i)    # i: 9, 10, …, 13 : f=3
dq.display()
```

C:\WINDOWS\system32\cmd.exe

```
[f=6,r=5] ==> [7, 5, 3, 1, 0, 2, 4, 6, 8]
[f=8,r=2] ==> [3, 1, 0, 2]
[f=3,r=2] ==> [13, 12, 11, 10, 9, 3, 1, 0, 2]
```

우선순위 큐

우선순위 큐의 개념과 동작 원리

우선순위 큐의 개념

우선순위 큐

/ priority queue /

- 우선순위 개념을 큐에 도입한 자료구조
  - 모든 데이터가 우선순위를 가짐
  - 입력 순서와 상관없이 우선순위가 높은 데이터가 먼저 출력
  - 가장 일반적인 큐
- 응용 분야

도로에서 자동차 우선순위

네트워크 트래픽 제어

OS의 작업 스케줄링

...

우선순위 큐의 개념과 동작 원리

우선순위 큐 ADT

데이터

- 우선순위를 가진 요소들의 모음

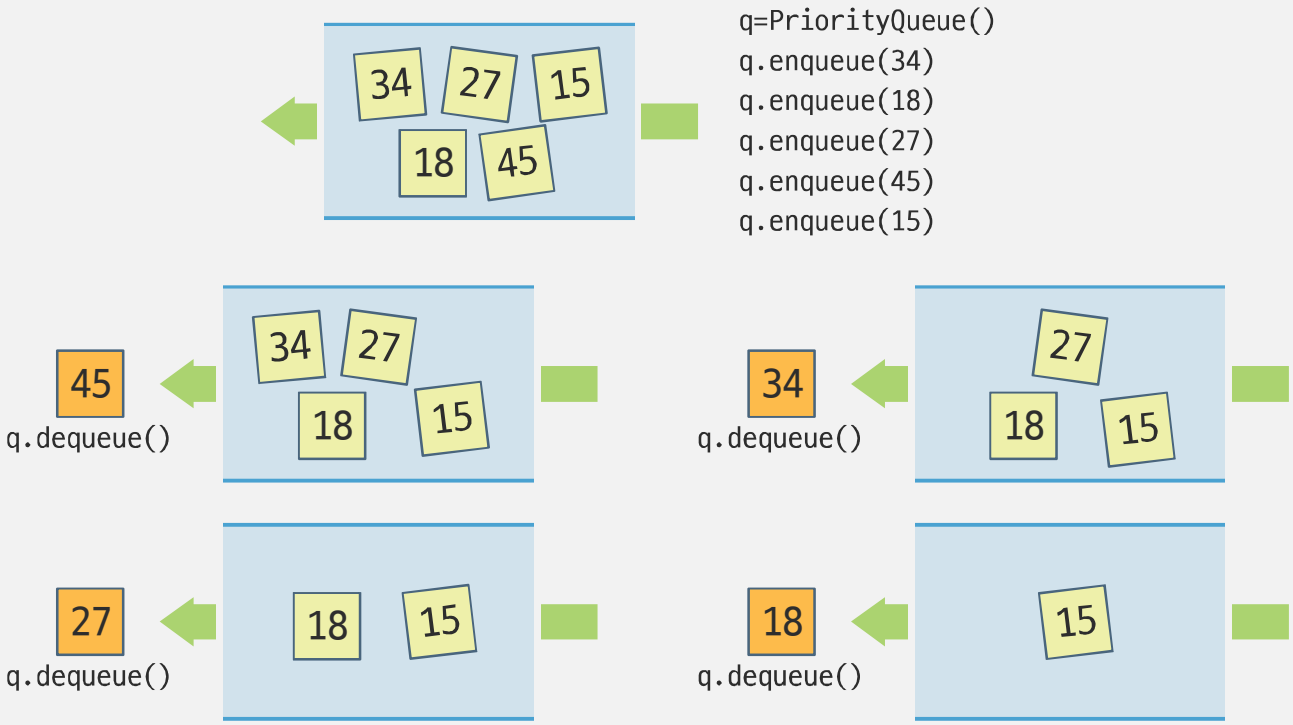
연산

- PriorityQueue(): 비어 있는 우선순위 큐를 만듦
- isEmpty(): 우선순위 큐가 공백상태인지를 검사함
- enqueue(e): 우선순위를 가진 항목 e를 추가함
- dequeue(): 가장 우선순위가 높은 항목을 꺼내서 반환함
- peek(): 가장 우선순위가 높은 요소를 삭제하지 않고 반환함
- size(): 우선순위 큐의 모든 항목들의 개수를 반환함
- clear(): 우선순위 큐를 공백상태로 만듦

우선순위 큐의 구현

우선순위 큐의 구현 방법

배열 구조 | 연결된 구조 | 힙 트리



우선순위 큐의 구현

정렬되지 않은 배열을 이용한 우선순위 큐 구현

```
# Python list를 이용한 Priority Queue ADT 구현

class PriorityQueue :
   def __init__( self ):                        # 생성자
      self.items = []                           # 항목 저장을 위한 리스트

   def isEmpty( self ):                         # 공백상태 검사
      return len( self.items ) == 0
   def size( self ): return len(self.items)     # 전체 항목의 개수
   def clear( self ): self.items = []           # 초기화

   def enqueue( self, item ):              # 삽입 연산
      self.items.append( item )            # 리스트의 맨 뒤에 삽입(O(1))
```

우선순위 큐의 구현

정렬되지 않은 배열을 이용한 우선순위 큐 구현

```
def findMaxIndex( self ):                # 최대 우선순위 항목의 인덱스 반환
    if self.isEmpty(): return None
    else:
       highest = 0                       # 0번을 최대라고 하고
       for i in range(1, self.size()) :# 모든 항목에 대해
          if self.items[i] > self.items[highest]:
          highest = i                    # 최고 우선순위 인덱스 갱신
       return highest                    # 최고 우선순위 인덱스 반환

def dequeue( self ):                     # 삭제 연산
    highest = self.findMaxIndex()        # 우선순위가 가장 높은 항목
    if highest is not None :
       return self.items.pop(highest)    # 리스트에서 꺼내서 반환

def peek( self ):                        # peek 연산
    highest = findMaxIndex()             # 우선순위가 가장 높은 항목
    if highest is not None :
       return self.items[highest]        # 꺼내지 않고 반환
```

(2/2)

우선순위 큐의 구현

테스트 코드

```
q = PriorityQueue()
q.enqueue( 34 )
q.enqueue( 18 )
q.enqueue( 27 )
q.enqueue( 45 )
q.enqueue( 15 )
print("PQueue:", q.items)
while not q.isEmpty() :
    print("Max Priority = ", q.dequeue() )
```

C:\WINDOWS\system32\cmd.exe

```
PQueue: [34, 18, 27, 45, 15]
Max Priority = 45
Max Priority = 34
Max Priority = 27
Max Priority = 18
Max Priority = 15
```

입력한 순서대로 우선순위 큐에 들어감

리스트에서 가장 큰 값을 반환

우선순위 큐의 구현

우선순위 큐의 시간 복잡도

01 정렬되지 않은 리스트 사용

- enqueue() : 대부분의 경우 $O(1)$
- findMaxIndex() : $O(n)$
- dequeue(), peek() : $O(n)$

02 정렬된 리스트 사용

- enqueue() : $O(n)$
- dequeue(), peek() : $O(1)$

03 힙 트리

- enqueue(), dequeue(): $O(logn)$
- peek() : $O(1)$

우선순위 큐의 구현

우선순위 큐의 사례

허프만 코딩 트리

빈도가 작은 두 노드를 선택

Kruskal의 MST 알고리즘

MST에 포함되지 않은 간선 중
최소 가중치 간선을 선택

Dijkstra의 최단거리 알고리즘

최단거리가 찾아지지 않은
정점들 중 가장 거리가 가까운
정점 선택

인공지능의 A* 알고리즘

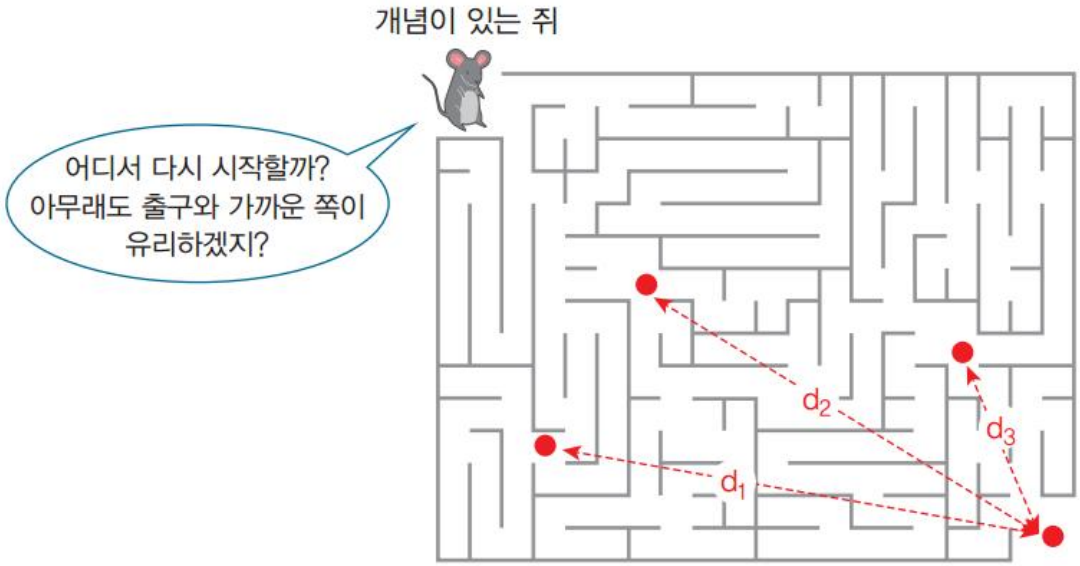
상태 공간 트리에서
가장 가능성이 높은 경로를
먼저 선택

우선순위 큐의 응용

전략적 미로 탐색

전략 (출구의 위치를 알고 있다고 가정)

가능한 한 가까운 방향을 먼저 선택



<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

우선순위 큐의 응용

전략적 미로 탐색

최대 우선순위 항목 선택

튜플 이용 (x, y, -d)

```
def findMaxIndex( self ):          # 최대 우선순위 항목의 인덱스 반환
    if self.isEmpty(): return None
    else:
        highest = 0                            # 0번을 최대라고 하고
        for i in range(1, self.size()):        # 모든 항목에 대해
            if self.items[i][2] > self.items[highest][2]:
                highest = i                    # 최고 우선순위 인덱스 갱신
        return highest                         # 최고 우선순위 인덱스 반환
```

우선순위 큐의 응용

전략적 미로 탐색

알고리즘

```
def MySmartSearch() :                          # 최소거리 전략의 미로탐색
    q = PriorityQueue()                        # 우선순위 큐 객체 생성
    q.enqueue((0,1,-dist(0,1)))                # 튜플에 거리정보 추가
    print('PQueue: ')

    while not q.isEmpty():
        here = q.dequeue()
        print(here[0:2], end='->')             # (x,y,-d)에서 (x,y)만 출력
        x,y,_ = here                           # (x,y,-d)에서->(x,y,_)
        if(map[y][x] == 'x'): return True
        else:
            map[y][x] = '.'
            if isValidPos(x, y - 1) : q.enqueue((x,y-1, -dist(x,y-1)))
            if isValidPos(x, y + 1) : q.enqueue((x,y+1, -dist(x,y+1)))
            if isValidPos(x - 1, y) : q.enqueue((x-1,y, -dist(x-1,y)))
            if isValidPos(x + 1, y) : q.enqueue((x+1,y, -dist(x+1,y)))
        print('우선순위큐: ', q.items)
    return False
```

우선순위 큐의 응용

전략적 미로 탐색

실행결과

C:\WINDOWS\system32\cmd.exe

```
PQueue:
(0, 1)->우선순위큐: [(1, 1, -5.0)]
(1, 1)->우선순위큐: [(1, 2, -4.47213595499958)]
(1, 2)->우선순위큐: [(1, 3, -4.123105625617661), (2, 2, -3.605551275463989)]
(2, 2)->우선순위큐: [(1, 3, -4.123105625617661), (3, 2, -2.8284271247461903)]
(3, 2)->우선순위큐: [(1, 3, -4.123105625617661), (3, 1, -3.605551275463989), (3, 3, -2.23606797749979)]
(3, 3)->우선순위큐: [(1, 3, -4.123105625617661), (3, 1, -3.605551275463989), (3, 4, -2.0)]
(3, 4)->우선순위큐: [(1, 3, -4.123105625617661), (3, 1, -3.605551275463989), (4, 4, -1.0)]
(4, 4)->우선순위큐: [(1, 3, -4.123105625617661), (3, 1, -3.605551275463989), (5, 4, -0.0)]
(5, 4)-> --> 미로탐색 성공
```

출구까지의 거리가 가장 가까운 위치부터 탐색

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	
1	큐에서 사용되는 정보의 입출력 방법으로 옳은 것은?	① FIFO ② LIFO ③ FILO ④ LILO	
2	큐의 응용 분야로 가장 옳지 않은 것은?	① 프린터와 컴퓨터 사이의 인쇄 작업 큐(버퍼링) ② 서브루틴의 복귀번지 저장 ③ 실시간 비디오 스트리밍에서의 버퍼링 ④ 시뮬레이션의 대기열	
3	선형 큐의 특징으로 가장 옳지 않은 것은?	① 삽입 시 불필요한 항목의 이동이 필요한 경우 발생 ② rear가 배열의 맨 끝을 가리킴 ③ 전단 삭제 시 , front 값이 증가 ④ 전단 삽입도 가능	
4	원형 큐의 특징으로 가장 옳지 않은 것은?	① 배열을 원형으로 사용 ② 인덱스만 회전시킴 ③ 포화상태일 때 배열의 값이 가득 차 있음 ④ 공백상태일 때, front와 rear가 같은 값	

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	
5	큐의 활용 분야로 가장 옳지 않은 것은?	① 기수정렬에서 레코드의 정렬을 위해 사용 ② 이진 트리의 레벨 순회 알고리즘 ③ 그래프의 탐색에서 너비 우선탐색 알고리즘 ④ 그래프의 탐색에서 깊이 우선탐색 알고리즘	
6	덱에 대한 설명으로 가장 옳지 않은 것은?	① 양쪽 끝에서 추가와 삭제가 가 ② 배열 구조로도 구현 가능 ③ 삽입 순서와 상관없이 우선순위가 높은 항목이 먼저 출력 ④ 선형 자료구조	
7	원형 큐를 상속하여 원형 덱 클래스를 만들 때 새로 구현해야 하는 연산으로 옳은 것은?	① size( ), clear() ② addFront(), deleteRear(), getRear() ③ isFull( ), isEmpty( ) ④ addRear(), deleteFront(), getFront()	
8	우선순위 큐의 특징으로 가장 옳지 않은 것은?	① 선형 자료형임 ② 우선순위의 개념을 큐에 도입한 자료구조 ③ 입력 순서와 상관없이 우선순위가 높은 데이터가 먼저 출력 ④ 모든 데이터가 우선순위를 가짐	
9	우선순위 큐 응용분야로 가장 옳지 않은 것은?	① 시뮬레이션 ② 네트워크 트래픽 제어 ③ 함수 호출 ④ OS작업 스케줄링	

학습정리

큐와 덱

큐(Queue)

- 큐는 자료의 입출력이 선입선출인 선형 자료구조의 일종임
- 큐의 전단은 삭제가 일어나며, 후단은 삽입이 일어남
- 큐의 연산으로는 Queue(), dequeue(), enqueue(e), peek(), isEmpty() 등이 있음
- 선형 큐는 삽입 시 불필요한 항목의 이동이 필요한 경우가 발생하여 인덱스를 회전시키는 원형 큐로 이 문제를 해결할 수 있음
- 원형 큐에서 front는 최근 삭제가 일어난 위치, rear는 최근 삽입이 일어난 위치임

덱(Deque)

- 덱은 전단과 후단 모두 삽입과 삭제가 가능한 입출력이 자유로운 자료구조임
- 덱의 연산으로는 Deque(), addFront(), deleteFront(), addRear(), deleteRear() 등이 있음
- 원형 덱은 큐와 데이터는 동일하고 연산은 유사함

우선순위 큐

- 우선순위 큐는 우선순위의 개념을 큐에 도입한 자료구조로 모든 데이터가 우선순위를 가지며, 입력 순서와 상관없이 우선순위가 높은 데이터가 먼저 출력됨
- 우선순위 큐의 연산은 배열 구조, 연결된 구조, 힙 트리 등으로 구현할 수 있음